Arduino Project One: Light

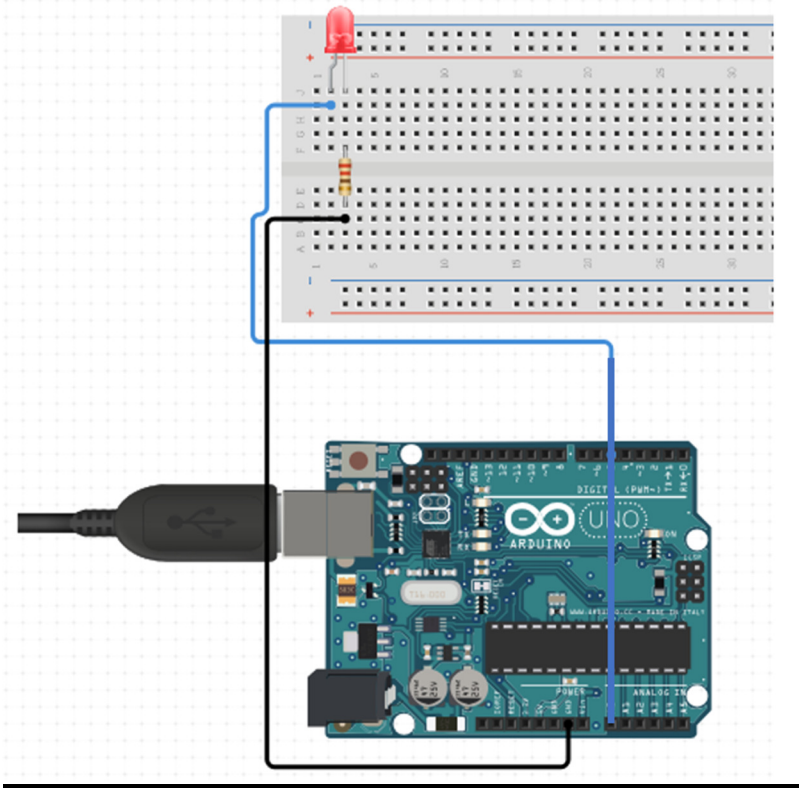
Activity 1F: Blink without delay (Multitasking)

Purpose: Become a clock-watcher! One simple technique for implementing timing is to make a schedule and keep an eye on the clock. Instead of a world-stopping delay, you just check the clock regularly so you know when it is time to act. Meanwhile the processor is still free for other tasks to do their thing. A very simple example of this is the BlinkWithoutDelay example sketch that comes with the IDE.

Materials: Computer, Arduino Uno Board, USB cable, Breadboard, 1 green LED, 1 yellow LED, 1 red LED, 3 - 220 Ω resistors.

New Ideas and Materials: What's the point of this? At first glance, BlinkWithoutDelay does not seem to be a very interesting sketch. It looks like just a more complicated way to blink a LED. However, BinkWithoutDelay illustrates a very important concept known as a **State Machine**. It literally allows the Arduino to walk and chew gum at the same time.

Circuit: Ready to start hooking everything up? Check out the Fritzing diagram and hookup table below, to see how everything is connected.



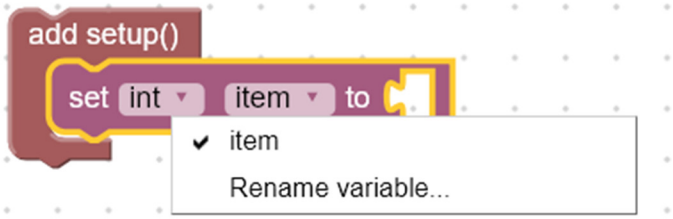
Pay special attention to the component's markings indicating how to place it on the breadboard. Polarized components can only be connected to a circuit in one direction.

Code

The code below uses the millis() function, a command that returns the number of milliseconds since the board started running its current sketch, to blink an LED.

1. You can create your code in the BlocklyDuino Editor then paste it to the Codebender website. Click BlocklyDuino Reference for a link to all the blockly codes.

2. First we need to create some variables. These are similar to X and Y you use in math class. **Variables** are used to store information to be referenced and manipulated in a computer program. They also provide a way of labeling data with a descriptive name, so our programs can be understood more clearly by the reader and ourselves. It is helpful to think of variables as containers that hold information. Their sole purpose is to label and store data in memory. This data can then be used throughout your program.
   Let's declare 2 variables to use to help us keep track of the time. You find the "add setup()" icon under "initializes in the menu and the integer variable under "Variables". Rename this using the dropdown to previousmillis.

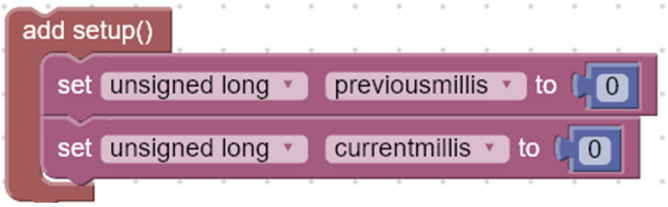


   "previousmillis" is not a simple integer but must allow for very large numbers so we use the "unsigned long" format. Also create another variable for "currentmillis" to track time

With funding from Canada

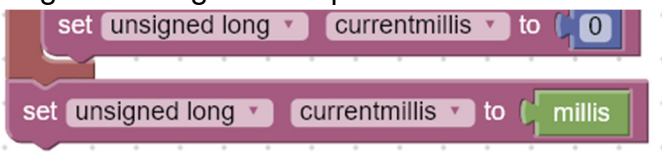
<SaskCode>

SIEC Saskatoon Industry Education Council

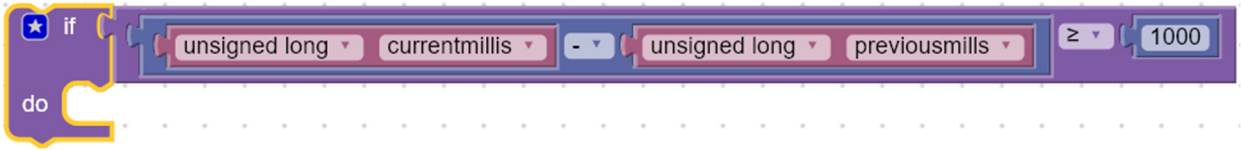


Note: We add a setup before the actual code begins. This means the Arduino will only read this information once before running the program. It helps speed up the program and is good coding practice.

3. Let's set currentmillis to the clock time from the chip. This will be updated each time the arduino goes through the loop.

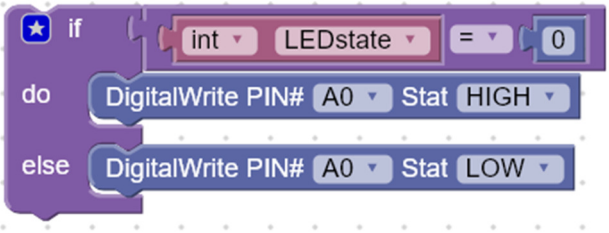


4. Now lets create the “if” loop to create a stopwatch. We will check the current time against the previous time and change the light to on or off in 100 ms intervals..



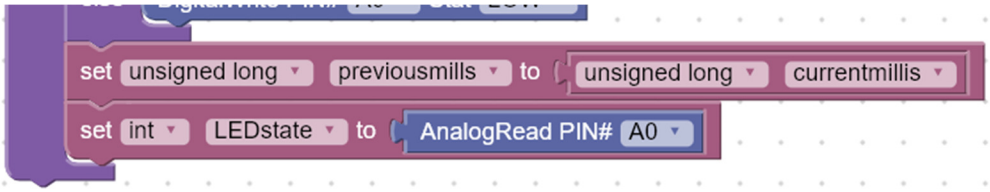
We are subtracting the current time minus the previous time equals 1000 milliseconds.

5. If this is true we will change the light. By using another if statement. If its off turn it on or vice versa.

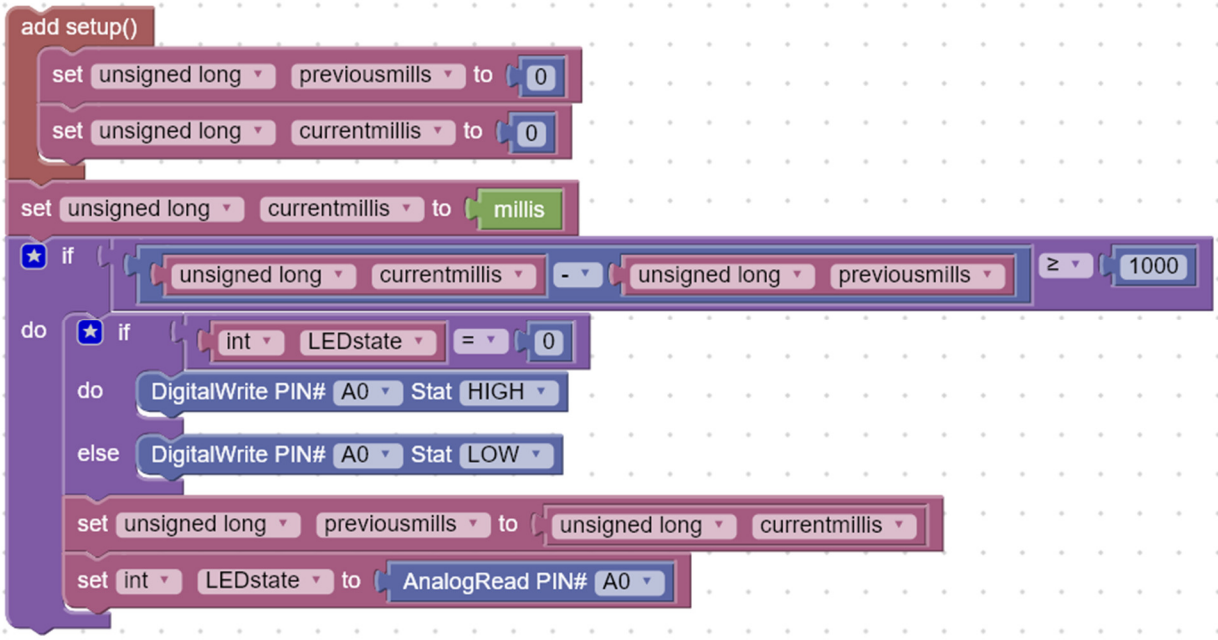


6. Finally lets reset our stopwatch to the current time and check to see if the light is on or off.

With funding from
Canada
<SaskCode>
SIEC
Saskatoon Industry Education Council



Here is the final product.



*****SAVE your sketch to your BlocklyDuino Folder that you created*****

And here is the code. You can now copy this by using “ctrl A” then “ctrl C”. Then paste this into the Codebender website.Select UPLOAD to program the following sketch onto your board.

With funding from
Canada
<SaskCode>
SIEC
Saskatoon Industry
Education Council

Arduino Uno

```
// This sketch was written by Bill Kapphahn at SaskCode.ca
  // Visit www.saskcode.ca
  // Version 1.0 5/2018

unsigned long currentmillis;
unsigned long previousmills;
int LEDstate;

void setup()
{
  previousmills = 0;
  currentmillis = 0;
  pinMode(A0, OUTPUT);
}


void loop()
{
  currentmillis = millis();
  if (currentmillis - previousmills >= 1000) {
    if (LEDstate == 0) {
      digitalWrite(A0, HIGH);

    } else {
      digitalWrite(A0, LOW);

    }
    previousmills = currentmillis;
    LEDstate = analogRead(A0);

  }
}
```

Your program is now loaded. Observe your LED and complete your worksheet.

Troubleshooting: If your circuit is not working!

1. Double check your code! Did you put in every block? Are the settings correct?
2. Refresh the website and reload the code.
3. The next step is to check your circuit!
    a. Check your wiring! Is it the same as the circuit diagram? Does the circuit go from the positive numbered pin to the ground?
    b. Next check each component. Check the leds, is the long positive arm connected by wires to the pins?
    c. It may be wise to test the led by turning them it on. Open a new blocklyduino window and create this test code. The block should look like this:



    d. LED still not working? Next inserting the LED directly into pin 13 and the adjacent ground. Does it light when you try this test code? If no



With funding from
Canada
<SaskCode>
SIEC
Saskatoon Industry Education Council

e. Next check the wires in the circuit. This can be done by connecting the LED to pin 13 only then using the wire to ground the short pin. Do not connect the pin to the ground directly. (Shorting the wires) This will damage the UNO.
f. You can also check the Uno itself by checking what signal is being sent to the pin using the serial monitor.
g. For further information on using the serial monitor see Activity 4A Using a Serial Monitor Display.

With funding from
Canada
<SaskCode>
SIEC
Saskatoon Industry Education Council

Activity 1F: Blink without delay (Multitasking)

1. Look carefully at the pins we used for the setup. Why do you think we used pin A0?

2. **THE CODE**

    a. Why is it good practice to add the Setup for variables?

    b. Delete the setup section and look at the arduino code. What do you notice BlocklyDuino does automatically for you?

    c. How could you speed up or slow down the blinking?

    d. You may have also noticed one if statement is inside the other if statement. We call this a "**nested**" statement. Why do you think that term is used?

Extension Challenges:

3. Change your code so that your LED flashes at 5 second intervals. Explain how you changed your code to achieve this.

4. Change your code and add an LED on another pin blinking at a different rate. You may have to introduce additional variables to achieve this.

5. Recreate a stop light. This time do it all without any pause statements. Try adding a push button to activate a walk light but not interfering with the street light.

Conclusion:

6. This type of code lets us run several functions at once without pausing the program. Why can the "pause" cause problems for multitasking?

With funding from
Canada
<SaskCode>
SIEC
Saskatoon Industry Education Council